FACT: Curating Learned Dynamics for MPPI When Most Logs Are Wrong

Apurba Bose[a,1], Soovadeep Bakshi[b,1,*], Jiachen Li[b], Dongmei Chen[b]

[a]*Independent researcher, San Jose, CA, 95118, USA*
[b]*Walker Department of Mechanical Engineering, The University of Texas at Austin, Austin, TX, 78712, USA*

Abstract

A dynamics model trained on consistently wrong logs can make MPPI confidently unsafe. When faulty logs form a majority in a regime, the logs alone cannot reliably identify which group matches the deployed robot. FACT (Failure-Anchored Curation of Training logs) uses a short trusted trajectory from that robot, typically a failure, as an attribution target. It removes every log whose mean gradient-alignment score opposes the reference and retrains, with no removal count to tune. On a quadrotor simulated from real NeuroBEM logs, with 67% of descent data faulty, one crash dive lowers the crash rate from 99% to 3.1%; robust fitting leaves 97–98%, and adding the crash leaves 71–95%. A supervisor's takeover also supplies the reference. A gradient-free score obtains similar crash rates, but gradient-based curation improves safe completion on the look-alike fault by 9.3 percentage points. A further 2,160 simulated dives on a thrust-gain fault test three obstacle-penalty settings with frozen corrected models: FACT has lower mean crash rates throughout, while learning to reweight completes more dives safely. The result is a safety–completion tradeoff, alongside FACT's lower measured correction cost and explicit curated dataset. On a battery fault in real Crazyflie logs, curation alone fails and three crashes help mainly as data; on a car whose model cannot represent ice, curation acts as a regime filter. All closed-loop tests are simulated.

Keywords: learned dynamics, model predictive path integral control, data curation, training-data attribution, failure data, quadrotor

*Corresponding author.
Email address: soovadeep.bakshi@utexas.edu (Soovadeep Bakshi)
[1]These authors contributed equally to this work.

1. Introduction

Learned dynamics models let sampling-based model predictive control such as MPPI [1, 2] fly and drive aggressively, because the planner can exploit effects that a first-principles model misses [3, 4, 5, 6, 7, 8, 9]. The same exploitation makes the planner fragile: where the model is optimistic, MPPI plans to the edge of what the model allows, and the robot crashes.

A model is optimistic where its logs are wrong, and system-identification logs are often wrong in a consistent way. A fleet's logs pile up across propeller sets, firmware versions and batteries, and a log recorded before a propeller change is a clean recording of a slightly different robot. If such logs make up most of the data in some regime, the learned model is confidently wrong there.

Standard data cleaning trusts the majority: robust losses and small-loss filtering discard what disagrees with the consensus, and models trained on a consistent majority agree, so ensembles see no problem. When faulty and correct logs cover the same states, no rule that sees only the logs can reliably identify which group matches the deployed dynamics (Proposition 1). Data from the deployed robot breaks the tie, and a failure that has happened is such data, from where the model is wrong. But a few failure runs are a few hundred samples against tens of thousands of faulty ones: added at a fixed weight they are outvoted (Proposition 2), and they cover only the states the failure visited.

Our observation is that, for such faults, a failure is more useful as an answer key than as training data: it is a trusted record of the regime where the model is wrong, and any such record serves (Section 7.5). FACT scores every training sample by whether a gradient step on it would increase the model's error along the failure [10], removes every log with a positive mean score, and retrains (Fig. 1). On a quadrotor whose plant is fit to real NeuroBEM flight logs [7], with two thirds of the descent data recorded under a propeller fault, the planner crashes in 98.9% of dives. One crash dive, used by FACT, brings this to 3.1%, close to the 1.9% of a model trained without the faulty logs, while adding the same dive to training leaves 94.7% (Table 1). A run that a safety supervisor takes over before impact serves as well as a crash.

In an idealised setting the sign of FACT's score separates the two groups whichever is the majority (Propositions 3–5), and removing the wrong logs fixes the model wherever they covered, not only along the failure. The claim is deliberately narrow: FACT addresses consistent majority faults in learned dynamics,

where faulty and correct logs overlap in state space, using a few closed-loop failures as trusted references for the deployed dynamics; it is not general-purpose data cleaning. A large part of this paper is about where the claim stops. Several assumptions of the analysis fail in our experiments, and Section 8 maps each assumption to where it holds, where it fails and what FACT does there.
Our contributions are:

- An idealised account of why failure-free cleaning cannot remove a consistent wrong majority, why failures added at a fixed weight leave the fault where they do not reach, why a failure used as an attribution target separates the groups, with or without normalisation, and why a log's mean score is reliable where its samples are not; and a measurement of how far a trained network's attribution score is from that account (Section 4).

- FACT, failure-anchored curation of training logs: the failure's transitions are trusted labels of the deployed dynamics, each sample is scored once by gradient alignment with them, logs with a positive mean score are removed, with no count to choose, and the unchanged pipeline retrains, returning an explicit list of logs to inspect; a gradient-free ablation isolates the value of the trusted reference, while the look-alike fault identifies a setting where gradient-based scoring improves safe completion (Sections 5 and 7.2).

- Closed-loop experiments on a quadrotor whose plant is learned from real NeuroBEM logs, a MuJoCo car and a fault in real Crazyflie logs, against failure-free cleaning, location rules, CUPID [11], weighted addition, replay fine-tuning and learning to reweight [12], with budget and random-removal controls, other plants, degraded failure records and sweeps over the fault. An additional 2,160 paired evaluation dives with frozen models establish the observed safety–completion tradeoff across three controller-cost settings (Section 7).

- An assumption-by-assumption account of where FACT stops, and practical checks for deciding when to use it (Section 8).

Section 2 reviews related work, Section 3 states the problem, Section 4 analyses it, Section 5 gives the method, Section 6 the experimental setup, Section 7 the results and Section 8 the discussion; Section 9 concludes.

2. Related work

Learned dynamics for planning and control. Neural dynamics models drive model-based control from MPPI with learned models [2] and model-based reinforcement learning [3, 4] to agile quadrotor flight, where they capture ground effect [5], aerodynamic residuals [6, 7, 9] and wind [8]. Brunke et al. [13] survey how such models are made safe. These works assume that the logs record the deployed robot, and treat model error as a matter of capacity or coverage. We study what happens when most logs in a regime record a different robot.

Adapting to changed dynamics. Online adaptation and meta-learning adjust a model to current conditions from recent data [14, 8]. That corrects the model near the states the robot is visiting, which is what fine-tuning on a failure does; we find that this leaves the fault where the failure did not go (Section 7.3). Curation removes the wrong logs everywhere they covered, and the two are complementary.

Failure-free robustness. Probabilistic ensembles and uncertainty penalties keep a planner where several learned models agree [4, 15], but disagreement signals too little data, not wrong data: models trained on a consistent fault agree. Robust losses [16], small-loss sample selection [17] and confident learning [18], surveyed by Song et al. [19], identify bad labels by their disagreement with the rest of the data. They work when bad labels are a minority or inconsistent, as we confirm, and fail when they are a consistent majority.

Reweighting towards trusted data. Learning to reweight [12] weights each minibatch sample, at every step, by the alignment of its gradient with that of a small clean set; Meta-Weight-Net [20] learns the weighting instead. It is the closest approach to ours, and with a failure run as its clean set it reaches comparable crash rates to FACT on two of three benchmarks (Section 7.3). FACT differs in its trusted data (the deployed robot's own failure), its unit (whole logs, whose mean score is reliable where single samples are not, Proposition 5), its cost (one scoring pass) and its output (a list of logs to inspect, then ordinary retraining). Fine-tuning with replay of old data [21] is the standard way to adapt a model without forgetting; we compare against both.

Data attribution and curation in robot learning. Influence functions [22], TracIn [10], datamodels [23], TRAK [24] and data valuation [25] estimate how training samples affect a model's predictions. In control they have curated the demonstrations that train controllers for cyber-physical systems [26], attributed an identified linear model's LQR performance to its identification trajectories [27], curated the training data of a learned safety predictor for control barrier functions [28], selected the data columns of data-enabled predictive control [29], and

scored MPPI's own sampled rollouts rather than any training data [30]. In each, the score's target is a task or safety objective (constraint violations, an LQR cost, a safety loss or a control cost), and faulty data, where studied, are a minority (5–10% poisoned demonstrations in Li et al. [26]). FACT's target is instead the prediction error on a correctly labelled record of the deployed robot, typically its own failure; the data it curates are the logs that train a dynamics model for MPPI; and it addresses a consistent wrong majority, which no rule that sees only the logs can reliably identify (Proposition 1). In robot learning, data quality matters for imitation [31], and demonstrations are curated by mutual-information estimates of their quality [32], by datamodels of a target task's loss [33], or, in CUPID [11], by their influence on closed-loop success. CUPID can curate from failed rollouts alone (its TuckBox task starts at 0% success), but its practical score is label-free: it measures how much a demonstration resembles the rollouts in gradient space, so it sees where a failure happened, not whether a log's labels are wrong. FACT uses the failure as correctly labelled data: one run sufficed on our quadrotor benchmarks, with no removal count, and it tells faulty logs from correct ones in the same states, where CUPID from 100 rollouts, told to remove 40, crashes in 39–93% of dives against 1.4–3.1% (Section 7.1; our adaptation of CUPID to dynamics models is in Appendix B). Crashes have also been collected deliberately as negative examples for learning a policy [34]; we use a failure instead as a reference for the dynamics.

3. Problem formulation

A learned model f_θ maps an input x (state features and command) to a target y (here, the part of the body acceleration that a nominal model does not explain). It is trained by least squares on a set of logs $\mathscr{L} = \{\ell_1, \ldots, \ell_M\}$, each a sequence of transitions (x, y). MPPI plans with f_θ and controls the deployed plant, whose map is $f^\star$. A *failure run* τ is a closed-loop run that violates a safety constraint, or that a safety supervisor takes over before it does. Its transitions $(x_t, y_t^\star)$ are observations of the deployed dynamics, $y_t^\star = f^\star(x_t) + \text{noise}$. More generally, a *reference* $\tau = \{(x_t, y_t^\star)\}_{t=1}^T$ is any such correctly labelled record of the deployed robot; a failure run is the reference the planner produces by itself where its model is wrong, and Section 7.5 tests others. In our experiments the deployed plant is simulated; for the quadrotors it is a model fit to real flight logs.

Consistent fault. In some regime $\mathscr{R}$ (for a drone, fast axial descent), a set of logs $\mathscr{B}$ follows a map $g \neq f^\star$: $y = g(x) + \text{noise}$. The fault is *consistent*: the bad logs agree with each other, so a model can fit them. It is a *majority* fault if the bad

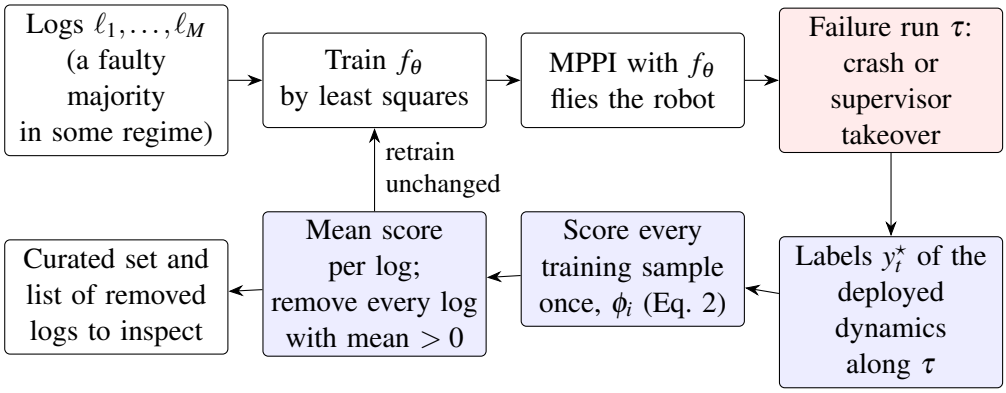


Figure 1: Failure-anchored curation. A failure of the planner (red) records the deployed dynamics where the learned model is wrong. FACT (blue) uses it as a reference, as it could any correctly labelled record of that regime (Section 7.5): it scores each training sample by whether training on it moves the model away from the failure, removes the logs whose mean score is positive, and retrains with the unchanged pipeline.

logs hold more than half of the data in $\mathscr{R}$. The goal is to find and remove $\mathscr{B}$ from a reference, typically a few failure runs, retrain, and run without crashing on new tasks.

4. Why a deployment reference is needed

4.1. Logs alone, and failures as data

Proposition 1 (Logs alone cannot reliably identify the wrong group)**.** *Let the logs in $\mathscr{R}$ come from two groups with label maps g_1 and g_2, sharing the same input distribution and noise, where group 1 holds a fraction ρ of the data. Consider world 1, where $f^\star = g_2$, and world 2, where $f^\star = g_1$. Any cleaning rule C that sees only the logs and outputs a model $\hat{f} = C(\mathscr{L})$ satisfies, at every $x \in \mathscr{R}$,*

$$\mathbb{E}_1 \|\hat{f}(x) - g_2(x)\| + \mathbb{E}_2 \|\hat{f}(x) - g_1(x)\| \;\geq\; \|g_1(x) - g_2(x)\|. \tag{1}$$

So in at least one world its expected error is at least half the gap between the groups. A rule that follows the majority has the full gap as its error in the world where the majority is wrong.

Proof. The logs are generated the same way in both worlds; only which map is the deployed robot differs. So $\hat{f}(x)$ has the same distribution in both worlds, and the bound is the triangle inequality applied inside one expectation. □

Robust losses, worst-fit filtering and ensembles are all such rules. Any correctly labelled data from the deployed robot in $\mathscr{R}$ breaks the symmetry; a failure is the record a planner produces on its own where its model is wrong (Section 7.5 compares it with other such records). Rules that see *where* a failure happened but not its labels, such as dropping the most crash-like logs or CUPID's label-free surrogate, find bad logs only if no correct look-alike logs cover the same states (Section 7.1).

Proposition 2 (A fixed weight leaves the fault where failures do not reach)**.** *Suppose the model can fit each input separately (a pointwise least-squares fit). At an input with N_b faulty and N_g correct samples, adding n failure transitions with weight W leaves a fraction $N_b/(N_b+N_g+Wn)$ of the fault $g-f^\star$ in the model. At an input the failure runs do not visit ($n=0$) the fraction is $N_b/(N_b+N_g)$ for every W. Removing the faulty logs leaves none at any input.*

Where a failure has data, a large enough weight does fix a pointwise fit; on our drone benchmark (27k faulty against 13k correct descent samples) one crash dive of 94 transitions at $W=20$ still leaves about 64% of the fault there. Larger weights are limited by coverage: how far a network spreads a heavily weighted fix is empirical (Section 7.3).

4.2. A failure as a reference

Proposition 3 (The score's sign separates the groups)**.** *In the setting of Proposition 1, with $\Delta = g_1 - g_2$, assume (i) the fit is pointwise least squares, $\hat{f} = \rho g_1 + (1-\rho)g_2$; (ii) the reference's residual comes from the fault alone, $e_t = \hat{f}(x_t) - f^\star(x_t)$; (iii) the fault keeps its direction over $\mathscr{R}$, $\langle \Delta(x), \Delta(x') \rangle > 0$; and (iv) the score uses a nonnegative scalar kernel, $\phi_i = -\sum_t k(x_t, x_i)\langle e_t, e_i \rangle$ with $e_i = \hat{f}(x_i) - y_i$, $k \geq 0$, and $k(x_t, x_i) > 0$ for some t. Then, in expectation over label noise, faulty samples have $\phi_i > 0$ and correct samples $\phi_i < 0$, for every $\rho \in (0,1)$.*

The sign does not depend on which group is larger: the failure fixes the reference, not the vote (proofs in Appendix A). FACT divides each score by the gradient norm, which can flip the sign of an expectation (Appendix A), but not with spherically symmetric noise, two or more outputs and an isotropic Jacobian.

Proposition 4 (Normalisation keeps the sign)**.** *In the setting of Proposition 3, let $\tilde{\phi}_i = \phi_i / \|\nabla_\theta l_i\|$ with $\nabla_\theta l_i = J_i^\top e_i$, and assume further (v) the outputs have dimension $d \geq 2$, the label noise has a spherically symmetric density, and $J_i J_i^\top = c_i I$ with $c_i > 0$. Then $\mathbb{E}\tilde{\phi}_i$ has the sign of $\mathbb{E}\phi_i$.*

Proposition 5 (A log's mean is reliable where its samples are not)**.** *Let a log's n samples have cosine scores $c_j = \phi_j/\|\nabla L_\tau\| \in [-1,1]$ with mean $\mu = \frac{1}{n}\sum_j \mathbb{E}c_j \neq 0$, each σ-sub-Gaussian about its mean ($\sigma \leq 1$), and τ-dependent: $\{c_j\}_{j\leq k}$ is independent of $\{c_j\}_{j\geq k+\tau}$ for every k. Then the log's mean score has the sign of μ except with probability at most $\exp(-n\mu^2/(2\tau\sigma^2))$, while a single sample's chance of the wrong sign does not shrink with n.*

The proof splits the log into τ interleaved runs of independent samples [35]. On our three quadrotor benchmarks (seed 1), 9–45% of samples and 2–36% of 2-s windows score with the wrong sign, but no bad flight and 0–6 of the 63 correct ones. Scores stay correlated for 0.3–0.9 s, so a window holds a few nearly independent samples and a flight 45–170. As a heuristic only (short correlation does not imply independence, and we set σ to the spread), each flight's statistics put the bound below 1% for 25–32 of the 32 bad flights. Hence one failure rates whole flights but not 2-s windows (Section 7.7). Averaging cannot fix a partly faulty log, whose μ may have the wrong sign, and per-sample reweighting [12] does not average.

4.3. From the idealised score to a trained network

These statements are idealised: a network's tangent kernel is neither scalar nor isotropic, (ii) fails when the model cannot represent the plant, and they describe a gradient step on one sample, not deleting logs and retraining (under (i) the two agree). We therefore test the unnormalised score, matched budgets and random removal (Section 7.3), and measure here how far the score of a trained network is from Proposition 3.

Write a flight's score as $\phi = \psi + \delta$, where ψ is the score of Proposition 3 with each 3×3 block $K_{ti} = J_t J_i^\top$ of the tangent kernel replaced by the closest block that meets (iv), $\max(\operatorname{tr}K_{ti}/3, 0)\,I$, and δ is what the rest of the kernel adds. If ψ has the predicted sign and $|\delta| < |\psi|$, so does ϕ. Over 35 drone runs (Tables 1 and 7), this condition holds for 88% of the bad flights, 83% of the look-alikes and 79% of the other correct flights with descent data, and ϕ has the predicted sign for 98%, 86% and 94%. Pair by pair the condition often fails (the scalar part is negative for 1–36% of pairs); averaging over the failure run and the flight is what makes δ small, and why short windows are not enough (Section 7.7). The margin shrinks where FACT is weaker: the median $|\delta|/|\psi|$ over bad flights is 0.13–0.24 on the benchmarks but 0.46–1.01 at +20% thrust, and the share that meets the condition tracks AUROC (Spearman 0.81). Of the 24 bad flights scored with the wrong sign, 13 already have the wrong ψ, so the non-scalar part of the kernel

is not what flips them: either assumptions (i)–(iii) fail for them, or finite-sample noise outweighs a sign that Proposition 3 guarantees only in expectation; the 33 wrong-sign look-alikes are all flipped by δ.

The analysis therefore explains FACT at the level of flights, not of samples or kernel entries, and its assumptions are best read as conditions to check. Section 8.1 collects where each one fails in our experiments.

5. Method

FACT's attribution score is TracIn [10]. The contribution is the combination around it: a trusted reference, by default the failure trajectory, supplies labels of the deployed dynamics, scores are aggregated per log, logs are removed by the sign of that mean without knowing how many are corrupted, and the unchanged pipeline retrains, with the result judged in closed loop (Algorithm 1). Learning to reweight [12] uses the same alignment to reweight every minibatch during training instead (Section 7.3).

Score. Let $L_\tau(\theta) = \frac{1}{T}\sum_t \|f_\theta(x_t) - y_t^\star\|^2$ be the mean squared prediction error of f_θ on the reference τ (Section 3), by default the transitions of one or more failure runs, and $l_i(\theta)$ the loss on training sample i. A gradient step of size η on sample i changes L_τ by about $-\eta\langle\nabla L_\tau, \nabla l_i\rangle$ [10]. We score each sample by

$$\phi_i = -\frac{\langle \nabla_\theta L_\tau(\theta), \nabla_\theta l_i(\theta)\rangle}{\|\nabla_\theta l_i(\theta)\| + 10^{-12}}, \tag{2}$$

positive when training on i pushes the model away from the reference (a sample with zero gradient scores zero); the normalisation keeps samples with large gradients from dominating a log's mean. For squared loss $\nabla_\theta l_i = J_i^\top e_i$, so $\langle \nabla L_\tau, \nabla l_i\rangle = \sum_t e_t^\top J_t J_i^\top e_i$: TracIn is the score of Proposition 3 with the network's tangent kernel $J_t J_i^\top$. Replacing that kernel by the identity gives a gradient-free variant, $\phi_i = -\langle \bar{e}_\tau, e_i\rangle$, which is the score of Proposition 3 with $k \equiv 1$ up to a positive factor and needs only forward passes; Section 7.2 compares the two.

Labels and removal. The reference's targets $y_t^\star$ come from its recorded states and commands through the pipeline that produced the training targets. A log's score is the mean of its samples' scores. We remove every log with a positive score (the *sign rule*), which needs no count, and retrain from scratch on the remaining logs with the original schedule, optionally with the reference's transitions added at weight $W = 20$. For comparison we also remove the top k logs, with k set from the number of bad logs, which a real fleet would not know. We score

Algorithm 1 FACT: failure-anchored curation of training logs

Require: logs $\mathscr{L} = \{\ell_1, \ldots, \ell_M\}$; the model f_θ trained on them, which the planner used; a trusted reference τ: one or more failure runs, or another correctly labelled record of the deployed robot in the regime where the model is wrong; the training pipeline TRAIN

recover the targets $y_t^\star$ of every transition of τ from its recorded states and commands, with the map that produced the training targets

$g \leftarrow \nabla_\theta L_\tau(\theta)$, with $L_\tau = \frac{1}{T}\sum_t \|f_\theta(x_t) - y_t^\star\|^2$ on the T transitions of τ

for every training sample i **do**

    $\phi_i \leftarrow -\langle g, \nabla_\theta l_i(\theta)\rangle / (\|\nabla_\theta l_i(\theta)\| + 10^{-12})$ ▷ one backward pass per batch

end for

$s_\ell \leftarrow \text{mean}_{i\in\ell}\, \phi_i$ for every log ℓ

$\mathscr{D} \leftarrow \{\ell \in \mathscr{L} : s_\ell > 0\}$ ▷ sign rule: no removal count

$f_{\theta'} \leftarrow \text{TRAIN}(\mathscr{L} \setminus \mathscr{D})$, optionally with the transitions of τ added at weight W

return $\mathscr{D}$, the logs to inspect, and $f_{\theta'}$

the final model, the one the planner used, rather than summing over checkpoints (Section 7.2 shows this changes little).

Cost. Scoring is one pass over the data. For a linear layer the per-sample gradient is the outer product of the layer's input and output gradients, so the norms and inner products in Eq. (2) need one backward pass per batch. Retraining costs at most one ordinary training. Section 7.3 reports wall-clock times.

What a failure must record. A failure need not be a crash: a run that a safety supervisor takes over is enough, provided the record includes the attempt to recover, which is where the model's error shows (Section 7.4).

6. Experimental setup

Quadrotor from real logs. The plant is a neural network fit on all 95 flights (75 min, four days) of the NeuroBEM dataset [7]. It predicts the body specific force beyond a thrust model, from body velocity, body rates and collective thrust. MPPI (512 samples, 0.8 s horizon) commands collective thrust and body rates with a 60 ms delay; each test flies 24 dives that end close to the floor, plus 24 steeper and 24 shallower dives, and a crash is any contact with the floor or walls (Appendix B). The planner's model is trained on the logs, some of which we make faulty. How the logs' targets are made, and how little FACT depends on that, is described in Appendix B.

Faults. Each fault touches the 32 flights with the most axial descent (8 per day), which hold 67% of the descent data. In the *propeller* fault those flights show 50% more thrust in axial descent, as logs from an older propeller set would; in the *thrust-gain* fault, 40% more thrust per command in every regime, as with a miscalibrated thrust map. In the *look-alike* version, of the 12 most-descending flights per day two in three are faulty and one in three is correct: 32 faulty flights (59% of the descent data) and 16 correct look-alikes (24%).

Protocol. On each benchmark and seed, the failure runs are the first crashing runs of the faulty model's planner on 24 attribution dives, every fix is evaluated on a disjoint set of dives, and every model is retrained from scratch with the seed's initialisation. We report the share of runs that crash and that reach the goal (within 0.5 m), since a planner can avoid crashes simply by being overly conservative. Reach counts any pass within 0.5 m, including dives that then crash, so a faulty model's reach is mostly dives that pass the goal on their way into the floor; reaching the goal without crashing is reported in Section 7.3 and Appendix B; means are over seeds with $\pm$ one standard error, and ranges are over seeds. Appendix B lists what each model is fit on.

Baselines. Without failures: Huber and L1 losses, removing the $k = 40$ worst-fit logs (32 are bad), a 5-model ensemble, and the ensemble with a disagreement penalty in the MPPI cost. With the location of the crash but not its labels: removing the k logs with the most descent, removing all descent data, and CUPID [11], adapted to a learned dynamics model and MPPI (Appendix B), with its label-free surrogate and with REINFORCE through MPPI, from 100 rollouts. With the failure runs: adding them to training, and the reweighting and fine-tuning of Section 7.3. The *oracle* removes exactly the bad logs.

Other platforms. A MuSHR car in MuJoCo with an icy corner (Section 7.8) and a battery fault in real Crazyflie logs (Section 7.9) test settings where the analysis's assumptions fail.

Implementation. Every learned model is an MLP with two hidden layers of 128 SiLU units, trained with Adam (details in Appendix B), in PyTorch on CPU.

7. Results

7.1. When most logs are wrong, a failure helps as a reference, not as data

Table 1 shows the main result. With most of the descent data wrong, the model crashes in 91–99% of dives, and every failure-free fix stays at 92–98%. Worst-fit filtering removes 22 of the 32 bad flights on the propeller fault, but the ten that remain keep the model optimistic. A 5-model ensemble crashes as often (100%,

Table 1: Quadrotor with most descent logs faulty: share of dives that crash (%, ± s.e. over 5 seeds, 3 for look-alikes) and that reach the goal (%; including dives that pass within 0.5 m of it and then crash), over test, steep and shallow dives. Fail.: failure runs the fix uses. CUPID, adapted to a dynamics model and MPPI (Appendix B), removes $k = 40$. The sign rule needs no removal count; the last row needs it.

		Propeller		Thrust gain		Look-alike	
Fix	Fail.	Crash	Reach	Crash	Reach	Crash	Reach
None (all logs)	0	98.9±0.5	34†	91.1±1.4	71	94.4±0.8	48
Oracle (bad logs removed)	–	1.9±0.7	64	1.9±0.7	64	0.0±0.0	75
Huber loss	0	97.8±0.7	34†	95.6±1.2	65	96.8±0.5	38
L1 loss	0	98.3±0.7	35†	94.2±0.9	65	95.8±0.8	35
Remove k worst-fit logs	0	97.2±0.4	36†	92.5±1.1	75	97.7±0.9	38
Drop k most crash-like logs	0§	15.8±4.4	89	44.2±4.1	89	88.0±2.3	51
Drop all descent data	0§	98.9±0.5	35	76.7±4.5	80	99.1±0.5	37
CUPID (adapted), label-free	100	54.4±3.5	76	–	–	39.4±16.2	58
CUPID (adapted), REINFORCE	100	78.9±19.4	39	–	–	92.6±6.1	38
Add crash dives, $W = 20$	1	94.7±3.6	36†	74.7±2.9	89	81.0±5.2	49
	3	69.4±3.8	57†	46.7±3.0	91	26.4±0.8	64
	24	20.6±1.7	67†	8.8±1.7‡	93	10.2±1.2	68
FACT, sign rule	1	**3.1±1.2**	61	**1.9±0.7**	64	**1.4±0.8**	79
	3	2.5±0.5	64	1.9±0.7	64	0.5±0.5	71
FACT, sign rule, then add, $W = 20$	1	2.5±0.5	65	1.9±0.3	68	0.9±0.9	78
FACT, supervised run, sign rule	1¶	3.1±0.7	61	1.9±0.7	64	1.4±0.0	75
FACT, remove top k (count known)	1	2.2±0.3	65†	3.1±0.8	63	0.5±0.5	79

§Uses only where crashes happen (descent). ¶Taken over before impact (Section 7.4). †Reach over seeds 3–5; the runs for seeds 1 and 2 recorded crashes only. ‡3 seeds.

3 seeds); with a disagreement penalty it crashes in 63% of dives at strength 1 and stops crashing at strength 10 only by never reaching the goal. From one crash dive and with no removal count, the sign rule removes 32–47 flights, 31–32 of them bad, and brings the crash rate to 1.4–3.1%, close to the oracle, reaching the goal about as often (61–79% against 64–75%); knowing the count does not help (0.5–3.1%). Adding the same crash dive to training barely helps (75–95%), and even all 24 crash dives leave 9–21% crashes (Fig. 2).

Look-alike logs and crude rules. On the propeller benchmark the bad flights are also the flights with the most descent, so dropping the 40 most crash-like flights already gets 15.8%, the loophole Proposition 1 warns about. With correct look-alikes mixed in, the same rule crashes in 88% of dives and CUPID's label-

Table 2: The principal fixes from one crash dive, re-flown with every dive recorded: dives that crash and dives that reach the goal without crashing (safe completion), in % (± s.e. over 5 seeds, 3 for look-alikes), and the wall-clock time of the whole fix, from the logs and the crash record to a trained model (propeller, seed 1, one CPU thread; an ordinary training on all logs takes 11 s). Rounding differs between machines, so re-flown crash rates can differ from those of Tables 1 and 3 by about one dive per seed.

	Propeller		Thrust gain		Look-alike		Time
Fix (one crash dive)	Crash	Safe	Crash	Safe	Crash	Safe	(s)
None (all logs)	98.9±0.5	1.1±0.5	91.1±1.4	8.9±1.4	94.4±0.8	5.6±0.8	–
Oracle (bad logs removed)	1.9±0.7	64.2±2.8	1.9±0.9	63.6±3.3	0.0±0.0	74.1±1.2	–
Add crash dive, $W = 20$	94.7±3.6	3.9±2.2	75.0±2.9	25.0±2.9	81.0±5.2	16.2±4.1	11
FACT, sign rule	3.1±1.2	60.3±2.6	1.9±0.9	63.6±3.3	0.9±0.5	78.7±3.3	7
2,640 updates	4.7±0.7	51.7±3.4	3.1±0.9	56.7±2.0	1.4±0.0	77.3±7.6	12
Gradient-free score, sign rule	3.9±0.5	64.7±4.7	1.9±0.9	63.6±3.3	1.4±0.8	69.4±3.7	6
Learning to reweight, look-ahead	2.5±0.5	83.1±4.1	9.2±0.7	83.3±1.5	1.4±0.8	80.6±5.0	31
per-sample gradients	2.8±0.8	76.1±5.3	10.0±2.0	80.0±3.7	0.9±0.5	79.2±6.4	443

free form in 39% (34% with $k = 32$). FACT still crashes in 1.4%, removing all 32 bad flights and at most two look-alikes (Section 7.2). In the REINFORCE form all 100 rollouts crash, so every return is -1, and with equal returns and no baseline the estimated return gradient is a sum of score-function terms whose expectation is zero. Its sign is noise: on the propeller fault it finds the bad flights on one seed of five and ranks them last on four (AUROC 0.99 against 0.01–0.08), and subtracting the mean return makes it exactly zero. This is a property of the exact policy gradient with equal returns, not of CUPID as published, which curates from failures alone with its surrogate.

Safe completion. Reach counts dives that pass the goal on their way into the floor, so Table 2 re-flies the principal fixes and counts safe completions, dives that reach the goal without crashing, with learning to reweight (Section 7.3) in the comparison. FACT completes about as many dives safely as the oracle on every fault (60–79% against 64–74%). Retrained for as many updates as the full-data model, it settles higher above the floor on the two faults without look-alikes (median final distance 0.73–0.78 m without a crash, against 0.64 m) and completes fewer (52–77%), crashing in 1.4–4.7%. Learning to reweight with the crash dive as its clean set completes more (76–83%), more even than the oracle, but on thrust gain it also crashes in 9.2–10.0% of dives, against 1.9%. Its planner ends closer to the goal and to the floor than the oracle's (median final distance without a crash 0.45–0.60 m against 0.64 m; closest approach 0.27–0.40 m against 0.43–

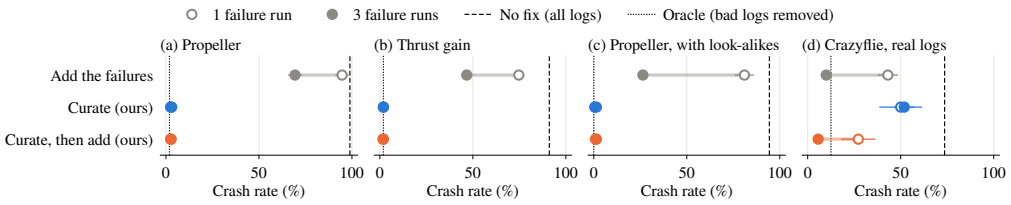


Figure 2: Dives that crash (%) after a fix using 1 (open) or 3 (filled) failure runs: add them ($W = 20$), FACT with the sign rule, or both. Mean $\pm$ s.e. over 5 seeds in (a), (b), (d), 3 in (c); where 1 and 3 runs coincide only the filled marker shows. Dashed: no fix; dotted: oracle. On the Crazyflie (d) curation alone fails; curating then adding three works.

0.46 m), and the goal lies below where a planner with a correct model settles (Appendix B). We read this as a smaller margin above the floor rather than a more accurate model; the terminal distances support that reading but do not establish it, and safe completion depends on where the goal lies relative to that margin. On thrust gain the smaller margin comes with more crashes. The trade-off is therefore explicit: reweighting completes more dives, while FACT crashes less on thrust gain, costs less and returns an explicit dataset. The two implementations of reweighting give the same weights up to rounding, which closed-loop flight amplifies into differences of a few dives. Adding the crash dive completes 4–25% of dives safely.

7.2. How the scores rate the flights

Figure 3 shows one look-alike run. By training loss the faulty flights look like the rest, since the model fits the majority: worst-fit filtering's top 40 holds 22 of the 32 faulty flights and 12 of the 16 look-alikes, and the last faulty flight ranks 92nd of 95. FACT's score from one crash dive is positive for all 32 faulty flights, no look-alike and six other flights, so the sign rule removes 38.

Count, checkpoint and rating. A removal count matters only when too small: on the propeller fault the top 24 leave 8 bad flights and crash in 70% of dives, while $k = 32$ to 64 gives 1.7–3.3% (5 seeds). Scoring at earlier checkpoints, or summing over all four as standard TracIn does, puts 31–32 bad flights in the top 40 on the propeller and gain faults. Rating a log by its top-scoring 2 s (the mean of its 200 highest sample scores, which need not be contiguous) instead of its mean changes little on the drone (Table 1: 2.2% against 3.1% on the propeller fault, 4.4% against 1.9% on the gain fault, with the sign rule's count), but the car needs it (Section 7.8).

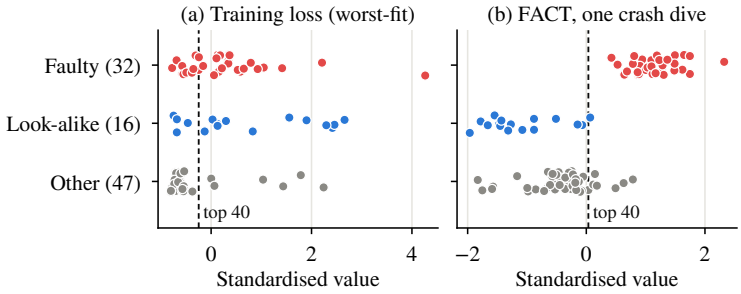


Figure 3: How each flight is rated on the look-alike benchmark (seed 1), one dot per flight; further right is more likely to be removed. (a) Training loss, used by worst-fit filtering. (b) FACT's score from one crash dive. Red: faulty; blue: correct look-alikes; grey: the rest.

Without the gradient. The gradient-free score uses the same crash dive, sign rule and retraining, but compares each sample's error with the dive's mean error instead of weighting the dive's states by the tangent kernel. On the quadrotor its crash rates are close to those of FACT (Table 2). It removes 40–41 flights on the propeller benchmarks, 30–31 of them bad (AUROC 0.97–0.99), and exactly FACT's 32 on thrust gain, and it crashes in 1.4–3.9% of dives against FACT's 0.9–3.1%. It needs only forward passes, and on the propeller benchmark it keeps 49% of the samples against 57%, so the whole fix takes 6 s against FACT's 7 s. These crash rates show that a trusted reference can support useful curation without a tangent kernel, consistent with Proposition 3, which covers this score ($k \equiv 1$). The tangent kernel matters where the model cannot represent the plant: on the car, told the count, FACT separates the bad logs from the look-alikes where the gradient-free score cannot (0.3% against 20.3%), and it finds 44–88 of a minority fault's 100 bad logs against 0–4 (Section 7.8). On the Crazyflie the two do the same (5.6% and 5.3%; Section 7.9).

The same stored runs also distinguish the scores by safe completion and selection. On thrust gain the two scores remove exactly the same 32 faulty flights and have identical stored outcomes. On the propeller fault, the gradient-free score removes 30–31 faulty and 10 correct flights, compared with 31–32 faulty and 1–5 correct flights for FACT; safe completion is 64.7% versus 60.3%. On the look-alike fault, FACT removes all 32 faulty flights in each seed, while the gradient-free score removes 31. Safe completion is 78.7% versus 69.4%, a paired advantage of 9.3 percentage points for FACT (95% interval $[0.6, 17.9]$ over three training seeds). Thus the gradient is not necessary for the thrust-gain result, but the look-

Table 3: Other uses of the crash dive, and budget and removal-size controls: crash (± s.e.) and reach (%), 5 seeds (3 for look-alikes). Models take the full-data 2,640 updates unless marked "fixed epochs" (12 passes). W: crash-dive weight (4,784: half of every batch). Fine-tuning starts from the faulty model at a 10× smaller learning rate.

	Propeller		Thrust gain		Look-alike	
Fix (one crash dive)	Crash	Reach	Crash	Reach	Crash	Reach
FACT, sign rule, fixed epochs	3.1±1.2	61	1.9±0.7	64	1.4±0.8	79
unnormalised score, fixed epochs	3.1±0.8	60	3.6±1.0	61	1.4±0.8	69
2,640 updates	4.4±0.7	52	3.1±0.9	58	1.4±0.0	78
As many random flights, fixed epochs	97.8±0.7	34	–		–	
2,640 updates	98.6±0.0	35	91.7±2.1	66	94.0±2.6	37
Add, $W = 20$, fixed epochs	94.7±3.6	36	74.7±2.9	89	81.0±5.2	49
Add, $W = 200$	89.2±7.0	43	–		–	
Add, $W = 2{,}000$	85.0±7.4	49	64.4±4.2	84	36.6±12.3	70
Add, balanced batches ($W = 4{,}784$)	79.4±8.4	50	67.2±5.0	85	28.2±7.2	69
Add, $W = 20{,}000$	81.9±11.5	44	–		–	
Fine-tune, half crash dive, half replay	84.4±14.5	39	83.1±2.9	69	19.4±6.4	65
Fine-tune on the crash dive only	70.6±16.8	44	–		–	
Learning to reweight, crash dive as clean set	2.8±0.8	77	9.7±1.7	86	2.3±0.9	79

alike results support its value for selection and completion. These are additional analyses of the existing runs in Table 2, not new evaluations.

7.3. Is it reweighting, extra training or less data?

Table 3 asks whether FACT's gain comes from its choice of logs, and whether the same crash dive could be used another way.

Budget and removal size. Removing as many random flights as the sign rule does leaves 92–99% crashes, with the original schedule or with as many updates as the full-data model, while FACT retrained for those 2,640 updates (about 18 passes over the kept data) gives 1.4–4.4%: neither less data nor a different training budget explains the fix. Scoring without the normalisation removes nearly the same flights (31–32 of the 32 bad on the propeller fault, all 32 on the others, no look-alike) and gives 1.4–3.6%.

Heavier weights and fine-tuning. On the propeller fault, raising the crash dive's weight from 20 up to balanced batches or $W = 20{,}000$ lowers the crash rate only to 79–89%, and fine-tuning the faulty model on the crash dive, with or without replayed logs, to 71–84% (one seed of five responds to all of these, 4–61%; the others stay at 76–100%); on thrust gain they leave 64–83%. The dive's 94

transitions correct the model along one trajectory, while new dives cross descent states the failure did not visit, where the faulty logs still dominate (Proposition 2). With look-alikes, correct flights cover those states and the faulty share is smaller (59% of descent data against 67%), and the same fixes do better (19–37%) but stay far above FACT's 1.4%.

Learning to reweight. With the crash dive as its clean set, learning to reweight is a second way to use the failure as a reference: it weights samples during training instead of removing logs. It matches FACT's crash rate on the propeller and look-alike benchmarks (2.8% and 2.3%), but crashes in 9.7% on thrust gain, against 1.9% (9.2–10.0% re-flown with either implementation, Table 2). Its weights shut the bad logs out (thrust error about 5 standard units on faulty descent data), but on thrust gain, where the bad flights are wrong in every regime, it also fits correct descent data worse (1.1–2.0 units, against 0.4–0.8). It completes more dives safely on all three faults, with a smaller margin above the floor (Table 2). The failure helps as a reference, not as data.

Cost. Table 2 gives the time of each complete fix on one CPU thread, from the logs and the crash record to a trained model (propeller benchmark; the other two take about as long, Appendix B). An ordinary training takes 11 s and adding the crash dive 11 s. FACT takes 7 s with its default schedule, of which 0.9 s is scoring, and 12 s with as many updates as the full-data model; with the gradient-free score the fix takes 6 s. Learning to reweight takes 31 s with the look-ahead of Ren et al. [12], 27 s with one forward-mode product per batch, and 443 s with per-sample gradients, the form behind Table 3. Learning to reweight therefore needs substantially more computation in our implementation, and it returns a model rather than a dataset.

7.3.1. Controller-cost sensitivity

To test whether the comparison is confined to the default controller cost, we vary only the obstacle-penalty multiplier over $\{0.5, 1, 2\}$ on the thrust-gain fault. For each of five training seeds, we train FACT and look-ahead reweighting once from the same faulty logs and the same first failing reference trajectory, obtained with the original controller. Both corrected models are then frozen across the three settings. The reference task seed is 11; evaluation uses task seed 7 and 24 test, 24 steep and 24 shallow dives per seed and setting, paired across methods and multipliers. The MPPI sample count (512), horizon (40), command delay (three steps), replanning interval (three steps), and all other costs are unchanged. This gives $5 \times 2 \times 3 \times 72 = 2{,}160$ new evaluation trajectories. We report every tested setting. Safe completion requires reaching within 0.5 m of the goal and no

Table 4: Controller-cost sensitivity on the thrust-gain fault. Rates are percentages, mean ± SE over five training seeds (72 paired dives per seed and setting). Each trained model is frozen across all three multipliers. Safe completion requires reaching the goal without a collision anywhere in the trajectory.

Multiplier	Method	Crash (%)	Safe completion (%)
0.5	FACT	5.83 ± 0.52	65.00 ± 1.61
0.5	Reweighting (look-ahead)	10.00 ± 1.35	85.56 ± 3.00
1	FACT	2.22 ± 0.56	63.33 ± 2.87
1	Reweighting (look-ahead)	8.06 ± 1.83	84.17 ± 3.69
2	FACT	1.39 ± 0.62	48.06 ± 1.79
2	Reweighting (look-ahead)	6.94 ± 0.62	80.00 ± 3.15

collision anywhere in the trajectory.

Table 4 and Fig. 4 show lower mean crash rates for FACT and higher mean safe-completion rates for reweighting at every multiplier. At the default multiplier, reweighting minus FACT is 5.83 percentage points in crashes (paired 95% interval $[0.46, 11.20]$) and 20.83 points in safe completion ($[13.22, 28.45]$). At multiplier 2 the corresponding differences are 5.56 ($[4.34, 6.77]$) and 31.94 ($[24.73, 39.16]$). Increasing the penalty reduces crashes for both methods but also reduces completion, especially for FACT. The observed tradeoff therefore persists across the tested controller range. In every seed FACT removes all 32 faulty flights and no correct flights, so its changing operating point comes from the controller cost, not a different curated dataset.

These runs use a macOS arm64 CPU with one PyTorch thread per worker; the default setting is rerun on that same host. They are compared within this experiment rather than pooled with Table 2's earlier runs. The intervals are paired Student-t intervals over training seeds, with each seed contributing its aggregate rate over 72 dives. The five seed aggregates, rather than individual dives, are the units of uncertainty.

7.4. Failures without a crash

Crashing a real drone is costly, so we also use runs that never crash. A supervisor watching the true state takes over the first time the drone could no longer stop above the floor with margin ($h < v^2/2a + 0.1v + 0.1$ m, with h the height, v the descent speed and a the spare thrust) and commands full thrust and level attitude. It saves every dive, about 0.7 s before impact. Scored from the first supervised

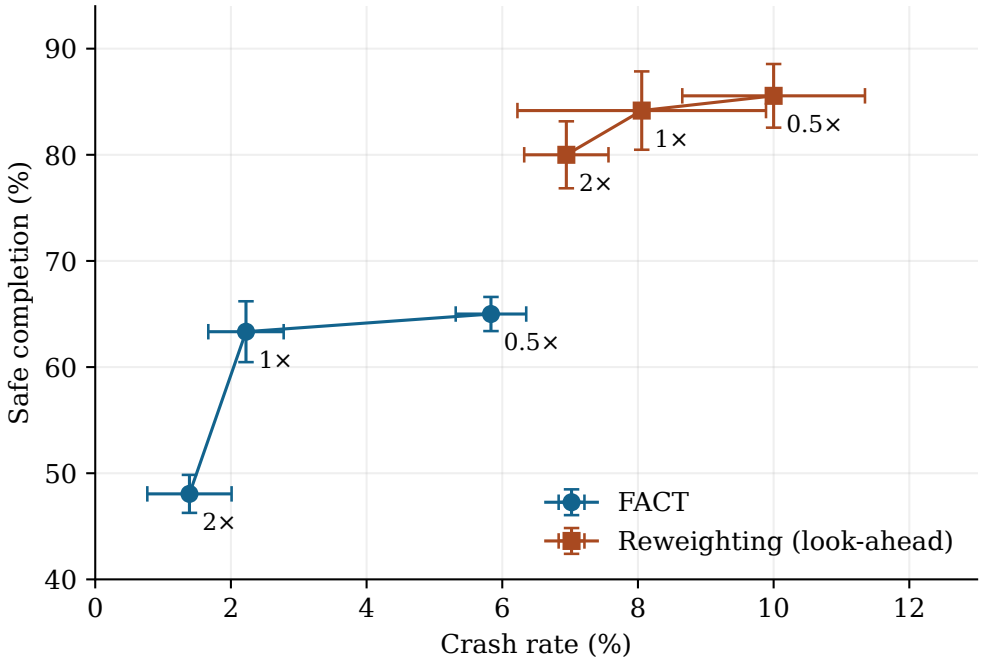


Figure 4: Safety and completion under three obstacle-penalty multipliers on the thrust-gain fault. Labels identify the multipliers; error bars are ±1 SE over five training seeds. Lines connect the tested settings of each frozen-model method. Lower crash rates and higher safe completion are desirable.

run through the takeover, the sign rule finds all 32 bad flights on every seed, removes no look-alike, and the planner crashes in 1.4–3.1% of dives (Table 1). The takeover matters: it is full thrust in fast descent, exactly where the propeller fault shows, while before it the faulty model brakes late at or below hover thrust, so that part alone finds only 18–31 of the 32 bad flights and crashes in $75.3_{\pm 10.0}$% of dives. A failure need not crash, but must record the attempt to recover.

7.5. What the reference must cover

Proposition 1 needs only correctly labelled data from the deployed robot in the faulty regime, and a failure is one such record. To separate what the failure contributes as a failure from what it contributes as coverage of that regime, Table 5 scores against four records of the same length: the crash dive; an expert dive, the crash dive's start and goal flown for as long in simulation by a planner that uses the plant as its model (an oracle-model controller standing in for a pilot or a conservative controller, not a human pilot); a routine flight, the faulty model's first

Table 5: The reference FACT scores against, each with the crash dive's number of transitions (93–99) and labelled by the plant. Desc.: share of the reference's transitions descending along the rotor axis faster than 1 m/s, where the propeller faults act (range over the three benchmarks). Error: the faulty model's mean squared error on the reference (normalised units). Found: bad flights removed (of 32) / flights removed (of 95), mean over seeds. Crash: dives that crash (%, ± s.e.), 5 seeds (3 for look-alikes); the crash-dive row is Table 2's FACT row.

		Propeller			Thrust gain			Look-alike		
Reference	Desc.	Error	Found	Crash	Error	Found	Crash	Error	Found	Crash
Crash dive	0.34–0.39	9.0	31.8/35.2	3.1±1.2	7.4	32/32	1.9±0.9	6.1	32/43	0.9±0.5
Expert dive	0.32–0.38	3.3	31.6/33.8	4.4±0.7	3.7	32/32	1.9±0.9	2.4	32/34.3	0.9±0.5
Routine flight	0.15–0.18	3.3	32/38	3.3±1.3	4.2	32/32	1.9±0.9	2.2	32/47.3	2.8±0.8
Hover	0.00	0.01	21.4/38.6	94.7±0.9	1.2	32/32	1.9±0.9	0.01	21.7/40	92.1±1.2

flight down the hangar that stops short of the end wall without crashing, thinned to the same number of transitions; and the faulty model holding its start position. The expert dive and the routine flight work about as well as the crash, finding all or nearly all 32 bad flights (crashes in 0.9–4.4% of dives, against 0.9–3.1%); the routine flight, with less than half the descent, removes more correct flights with the look-alikes (15 on average, against 11). Hover has no descent along the rotor axis, and on the propeller faults the faulty model already fits it (error 0.01, against 6.1–9.0 on the crash dive): it finds 21–22 of the 32 bad flights, and the planner crashes nearly as often as without a fix (92–95% against 94–99%). On thrust gain, a fault that acts in every regime, the faulty model is wrong in hover too, and hover finds all 32 (1.9%). What FACT needs is a correctly labelled record of the regime where the logs are wrong. A failure is the record the planner produces there by itself, without anyone knowing which regime that is; an ordinary flight that covers the regime, by design or incidentally as the routine flight does here, serves as well, and the faulty model's error on a candidate reference shows whether it reaches the fault.

7.6. Robustness

Other plants. With the plant that flies the failure run and judges every fix replaced by a wider tanh MLP, a smaller MLP fit on 70% of the flights, or gradient-boosted trees, FACT removes 31–32 of the 32 bad flights and stays within 0.5 points of the oracle, while adding the crash leaves 87–93% (Table 6, top). These plants fit the logs as well as the default plant (R^2 0.86–0.98 per axis) but differ from it by 0.17–0.29 m/s^2 on average. The mismatch plant adds to the default one a 30 ms motor lag, 1 m/s^2 gusts, 5% less thrust than commanded and a state

Table 6: Propeller benchmark under other conditions: dives that crash (%), 5 seeds (± s.e. for FACT). Top: another plant flies the failure run and judges every fix. Bottom: degraded failure records; Found: bad flights removed (of 32). Right: a milder fault (+35%) with 16 look-alikes (37% crashes unfixed). Under the mismatch even the oracle crashes in 22%: what remains is plant mismatch, not curation. The default row's FACT is 3.1 here and in Table 1, and 21.9 against 22.2 under the mismatch, because the two use the same sign rule and the oracle's crashes are the mismatch's. *Plant fit on 32 held-out flights; the logs are the other 63 (20 faulty), so no plant data reach the planner's model.

Plant	Unfixed	Add crash	FACT	Oracle
Default (Table 1)	98.9	94.7	3.1±1.2	1.9
MLP 3×256 tanh	98.9	93.1	1.4±0.4	1.4
MLP 2×64, 70% of flights	98.3	91.4	2.2±0.7	1.7
Boosted trees	98.9	87.2	1.4±0.8	1.7
Default + mismatch	98.1	95.6	21.9±1.4	22.2
Cross-fitted*	93.9	54.7	1.9±0.6	1.9

	Benchmark		+35%, look-alikes	
Failure record	Crashes	Found	Crashes	Found
Crash, as simulated	3.1±1.2	31–32	5.0±2.3	30–32
State noise ×1	3.1±1.2	31–32	4.7±2.4	30–32
×3	3.1±1.2	31–32	5.8±2.8	30–32
First 75% of the run	3.1±0.5	31–32	3.3±1.4	32
First 50%	62.8±15.7	15–31	16.7±3.3	19–25
Supervisor, margin 0.1 m	3.1±0.7	32	3.1±0.5	31
0.5 m	3.9±0.3	31–32	3.3±1.2	31
1.0 m	2.2±0.6	32	1.9±0.6	31
0.3 s reaction	2.8±0.6	32	1.9±0.3	31
0.5 m + noise ×1	3.9±0.3	31–32	1.4±0.0	31

estimate with correlated noise (2 cm, 0.05 m/s, 0.6°, 0.05 rad/s), which the planner and the failure record see. On every one of these plants FACT removes 31–32 of the 32 bad flights (33–36 in all). In a cross-fitted test the plant is fit on a third of the flights, every third by axial descent within each day, and the planner's logs are the other 63 flights, 20 of them faulty (64% of the descent data), so the data that define the plant never reach the planner's model. The plant predicts the unseen flights with R^2 0.84–0.96 per axis; FACT removes all 20 bad flights (20–38 in all) and matches the oracle on every seed (1.9%), while adding the crash leaves 55%. Under the mismatch, the oracle and FACT both crash in 22%: curation removes wrong logs but cannot supply dynamics that no log records.

Imperfect failure records. Logging the run from the noisy state estimate,

Table 7: Robustness on the propeller fault, one factor at a time from the benchmark (+50% thrust, 32 bad flights); sign rule from one crash dive, 3 seeds (5 for +20%, 44 bad and 32 look-alikes). Bad: share of descent data from bad flights. Removed: flights removed of 95 (bad / look-alikes). Crash rates (%): unfixed, FACT (± s.e.), oracle.

Setting	Bad	Removed	Unfixed	FACT	Oracle
Benchmark	67%	33–36 (32 / 0)	98.6	3.7±1.7	0.9
Thrust +20%	67%	39–55 (29–32 / 0)	10.3	3.3±0.8	1.9
+35%	67%	33–37 (31–32 / 0)	86.6	3.7±0.5	0.9
+80%	67%	33–35 (32 / 0)	100.0	4.2±1.6	0.9
Bad flights 12	32%	23–44 (12 / 0)	43.1	1.4±0.8	0.5
20	50%	27–31 (20 / 0)	92.6	0.5±0.5	0.9
44	80%	46–59 (42–44 / 0)	98.9	4.7±3.1	3.1
Plus look-alikes 8	61%	34–39 (32 / 0)	95.8	1.9±1.2	0.0
16	56%	40–60 (32 / 1–3)	90.3	0.9±0.5	2.3
32	49%	34–52 (27–30 / 0–11)	71.1	1.9±0.7	0.6

or from three times its noise, changes nothing (Table 6, bottom). Neither does a supervisor that takes over earlier (0.5 or 1.0 m margin) or reacts in 0.3 s, nor a noisy, supervised run with a milder fault and look-alikes. State noise and earlier or slower takeovers leave 1.4–5.8% crashes. With its first half only, the record misses most of the fast descent where the fault shows, finds 15–31 bad flights and leaves 63% crashes on average (17% with the milder fault).

Severity, faulty share and look-alikes. Table 7 varies the propeller benchmark one factor at a time (its first three seeds, hence 3.7 rather than 3.1 for the benchmark row). From +20% to +80% thrust, 12 to 44 bad flights (32–80% of the descent data) and up to 32 look-alikes, FACT stays within 3.3 points of the oracle (mean crash rates 0.5–4.7%, worst seed 17%), against 43–100% unfixed except under the mildest fault. At +20% thrust the crash run carries a smaller error, yet the sign rule still finds 29–32 of the 32 bad flights. With 44 bad flights (80% of the descent data), one seed of five misses two and crashes in 17%, the worst single seed of the study; the others crash in 0–4%. With 32 correct look-alikes the score separates less cleanly (AUROC 0.80–0.99): the sign rule misses up to five bad flights and removes up to 11 look-alikes, yet no seed crashes in more than 4.2% of dives, and the planner reaches the goal as often as the oracle's (56–96% against 57–97% of dives in these three settings).

Table 8: Fault that starts mid-flight: +50% thrust in descent from 25–50% into each of the 44 most-descending flights (63% of descent data faulty; 5 seeds). Fail.: crash runs used (the second from the curated model). Removed: faulty / correct descent data (%). Kept: data kept (%).

Rule	Fail.	Crash (%)	Removed	Kept
Unfixed	–	96.7±0.3	–	100
FACT, flights	1	37.5±17.8	70–98 / 27–43	42–52
FACT, flights	2	5.3±2.6	100 / 47–75	25–33
FACT, 2-s windows	1	60.6±11.7	93–98 / 3–5	47–49
Oracle, flights	–	3.1±0.7	100 / 46	47
Oracle, 2-s windows	–	14.7±2.2	99 / 2	68
Oracle, samples	–	1.1±0.3	100 / 0	68

7.7. A fault that starts mid-flight

So far a bad log is wrong from start to end. When the propeller fault instead starts 25–50% into each of the 44 most-descending flights, so that 21% of their descent data is still correct, one failure is not enough (Table 8): the flight mean mixes the two parts, the sign rule finds 37–43 of the 44 flights, two of five seeds crash in about 80% of dives, and the planner crashes in $37.5_{\pm 17.8}$% of dives on average (unfixed 97%). Rating 2-s windows locates the fault (AUROC 0.96–0.99 over windows with descent) but still crashes in 61% (single samples: 36%): a window has too few samples to average out label noise where the faulty model sits between the two groups (Proposition 5), and the planner is sensitive to what is left, since even removing exactly the mostly-faulty windows crashes in 15%. A second failure, a crash of the curated model, fixes the flight rule: it finds all 44 flights on every seed and crashes in $5.3_{\pm 2.6}$% (flight oracle 3.1%). The price is data: FACT removes the faulty flights whole, plus 14–26 correct ones, and keeps 25–33% of the data against 68% for an oracle that removes only the faulty samples. FACT works on whole logs: it finds the logs a fault enters partway but cannot separate their parts.

7.8. A car whose model cannot represent the plant

A MuSHR car [36] in MuJoCo [37] drives an L-shaped corridor whose corner is icy (friction 0.2); the learned model predicts 50 ms body-frame state changes, and we drive 24 runs from each of three entry conditions (test, late entry, entry at the ice edge). In the majority fault, 300 fast route runs recorded on a dry day make up 65% of the fast data over the ice, alongside 100 correct fast icy runs (look-alikes) and about 2,500 short exploration logs. The 300 faulty route runs

Table 9: MuJoCo car, most fast icy logs faulty: runs that crash (%, 5 seeds); bad logs (of 300) / look-alikes (of 100) removed.

Fix	Fail.	Crash	Bad / look.
None (all logs)	0	39.7±3.7	–
Oracle	–	0.0±0.0	300 / 0
Huber loss	0	41.7±5.2	–
L1 loss	0	77.8±2.8	–
Remove 360 worst-fit	0	48.1±7.6	0 / 22–32
Drop 360 most crash-like	0[§]	3.3±2.3	233 / 68
Drop all fast data on ice	0[§]	21.1±12.1	–
Add crash runs, $W = 20$	1	26.7±6.7	–
	3	10.3±4.6	–
FACT, sign count[†]	1	1.4±1.1	300 / 100[*]
	3	0.0±0.0	300 / 100[*]
Gradient-free, sign count[†]	1	0.8±0.6	300 / 100[*]
FACT, remove 360	3	0.3±0.3	283–300 / 50–60
Gradient-free, remove 360	3	20.3±15.9	229–288 / 63–100

[†]As many logs as have a positive mean score, ranked by their top-scoring 2 s. [§]Uses where crashes happen. [*]1,338–1,713 of 2,910 removed.

were recorded with dry-road friction, so they are wrong only over the ice. The fault crashes the planner in 40% of runs, and robust losses and worst-fit filtering do no better (Table 9).

Here we depart from the sign rule. The failure target is the last 2 s before contact, and logs are ranked by their top-scoring 2 s (the mean of their 40 highest sample scores), since the mean puts only 0–32 of the 300 bad logs in the top 360; the sign rule only sets how many to remove, the number of logs with a positive mean score (about 1,500 of 2,910). This fixes the planner from one crash run (1.4%), but not selectively. The car's model cannot represent the ice even with correct logs: without the faulty logs it still keeps 72–81% of the error on the crash runs, against about 1% for the drone. Assumption (ii) therefore fails, every score shifts up, and FACT removes about half of all logs, route runs and look-alikes included: a regime filter, which the gradient-free score matches (0.8%). The logs removed are the top-ranked ones: all 410 route runs, so all 100 look-alikes, and 1,000–1,200 exploration logs. The 1,300–1,800 samples of fast driving on the ice left in the kept exploration logs are what keep the planner off the wall. Told the count, FACT from three runs removes 283–300 of the bad logs and about half the look-alikes (0.3%), where the gradient-free score cannot tell them apart (20.3%);

Table 10: Crazyflie battery sag (real logs): crash and reach (%, 5 seeds). The battery voltage is logged but withheld from the planner's model. The oracle keeps only the low-voltage flights, the correct historical subset; the last row adds the same three crash dives to it.

Fix	Fail.	Crash	Reach
None (all flights)	0	73.6±3.4	24
Oracle (low voltage)	–	12.5±5.2	80
Huber loss	0	66.7±2.8	35
L1 loss	0	77.8±3.1	30
Remove 76 worst-fit	0	78.9±4.9	29
Remove 76 random	0	70.9±6.0	38
Drop 76 most crash-like	0[§]	80.3±7.3	26
Add crash dives	1	43.1±5.5	59
	3	10.0±3.0	87
Learning to reweight	1	49.4±14.9	50
	3	19.7±6.1	77
Learning to reweight + add	1	24.2±14.9	74
	3	10.0±5.4	91
FACT, sign rule	1	50.0±11.5	49
	3	51.9±5.5	49
FACT, sign rule + add	1	27.2±9.2	71
	3	**5.6±1.8**	92
Gradient-free, sign + add	3	5.3±1.8	94
Oracle + add	3	1.7±0.8	96

[§]Uses where crashes happen.

but dropping the 360 logs with the most fast driving on the ice does nearly as well (3.3%).

On a minority car fault (100 bad logs) the gradient-free score finds 0–4 of the bad logs against 44–88 for TracIn: where the model cannot represent the plant, correct logs near the failure have errors like the bad ones, and the tangent kernel, which weighs how training on a log changes the model at the failure states, separates them.

7.9. A fault in real logs: Crazyflie battery sag

NanoBench [38] logs flights of a Crazyflie 2.1 with motor commands and battery voltage; we use the 105 of its 107 public recordings that keep at least 2 s of airborne data (Appendix B); at low voltage the same command gives less thrust. The voltage is recorded in every log but deliberately withheld from the planner's model, as a stand-in for causes that are not logged, such as propeller wear; with the voltage as an input there would be nothing to curate (Section 8.2). The plant sees

the voltage and flies at 3.1 V, so the 76 flights flown at a median of 3.35 V or more (73% of the data) are consistently wrong for this battery. Every model takes equal gradient steps. Every fix without failures crashes in 67–80% of dives, against 12.5% for the low-battery flights alone (still above 3.1 V; Table 10). Unlike on the built faults, curation alone fails, from one crash or three (50–52%): the crash's error is as large sideways as vertical, so the score also reflects how much a flight resembles the dive (assumption (ii) fails), and the fault is a continuum (AUROC 0.35–0.63), so FACT does not reliably find the faulty flights here. Learning to reweight does better from three crashes (19.7%). Three crashes help mostly as data: adding them alone gives 10.0%, and curating with them first gives 5.6%, the same as the gradient-free score curated and added the same way (5.3%). This experiment therefore supports curating and then adding, not the gradient score. Given the same three crashes, the oracle's correct historical flights crash in 1.7%. The smaller differences are within noise. Paired over seeds, FACT plus adding minus adding alone is -4.4 points (95% CI -9.8 to 0.9; lower on four seeds, tied on one); minus reweighting and adding, -4.4 (-16.0 to 7.1; reweighting's worst seed crashes in 29% against 11%); minus the gradient-free score, $+0.3$ (-4.8 to 5.4); and minus the oracle with the crashes added, $+3.9$ (-3.3 to 11.0). With the plant fit on a held-out third of flights, curating and adding is again below adding alone (26% against 36%; oracle 52%).

7.10. Where failures are not needed

Our claim is limited to consistent faults that dominate a regime. With 12 propeller-faulty flights (a minority) among 12 correct look-alikes and 12 harmless noisy flights, an L1 loss crashes in 2% of dives and removing the 20 worst-fit flights in 5%, against 1.4% for FACT (5 seeds). With the motor log 100 ms late on the 32 most-descending flights, a majority but inconsistent fault, the lagged logs fit poorly and worst-fit removal fixes the model (4%, seed 1); FACT matches it only when each log is rated by its top-scoring 2 s of samples (2.8%, against 31% with the mean).

8. Discussion

8.1. Where the analysis holds and where it fails

Table 11 collects, for each condition of Section 4, the experiments where it holds and fails. Two patterns stand out. First, FACT degrades gracefully when a condition is only weakened: a shrinking margin under a mild fault, look-alikes that blur the separation, and noisy or earlier failure records all leave crash rates

Table 11: Each condition of the analysis (Section 4), where it holds and fails in our experiments, and what FACT does where it fails. Crash rates in %.

Condition	Holds	Fails	What happens
A consistent fault that dominates a regime (Section 3)	Every quadrotor and car majority fault; the Crazyflie battery fault	A minority fault; a motor log 100 ms late (inconsistent)	Failures are not needed: an L1 loss (2) or worst-fit removal (5; 4 for the lag) already works (Section 7.10)
(ii) The reference's error comes from the fault alone	Quadrotor: without the faulty logs about 1% of the crash runs' error remains	Car: 72–81% remains; Crazyflie: the crash's error is as large sideways as vertical	Car: a regime filter that removes about half of all logs (1.4); Crazyflie: curation alone fails (50–52), and the crashes added do most of the work (10.0 added alone, 5.6 curated and added)
(iii) Two groups, each fault of fixed direction	The built faults	Crazyflie: a voltage continuum, not two groups (AUROC 0.35–0.63)	As above
(iv) A nonnegative scalar kernel	After averaging over the failure and a flight: 88% of bad flights meet the margin condition	Pair by pair (scalar part negative for 1–36% of pairs); at +20% thrust the margin shrinks	The sign rule still finds 29–32 of 32 at +20% (Section 4.3); the gradient-free score, whose kernel meets (iv), crashes in 1.4–3.9 (Section 7.2)
(v) Normalisation keeps the sign	Not checked directly	–	Without the normalisation the score removes nearly the same flights (1.4–3.6)
Whole logs are faulty (Proposition 5)	The built faults	A fault that starts mid-flight	One failure: 37.5; a second failure: 5.3, keeping 25–33% of the data (Section 7.7)
The reference records the faulty regime	A crash, or a takeover with the recovery; an expert dive or a routine flight through the regime	The first half of the run; the supervised run before the takeover; hover	63, 75.3 and 92–95 (Sections 7.4, 7.5 and 7.6)
The logs record the plant up to the fault	Other learned plants, one fit on flights the planner never saw (within 0.5 points of the oracle)	Unlogged motor lag, gusts, thrust deficit and estimation noise	FACT and the oracle both crash in 22: curation cannot supply missing dynamics

within a few points of the oracle. Second, it stops working as curation when a condition fails outright: when the failure's error does not come from the fault, when a log is only partly faulty, or when the record misses the faulty regime. Most of these cases have a remedy within the same framework: adding the failures as data (Crazyflie, where they do most of the work), a second failure (mid-flight fault), or a record that includes the recovery (supervised runs). The car has none: FACT still fixes the planner there, but as a regime filter that a crude location rule nearly matches.

8.2. Using FACT in practice

The experiments suggest the following checks for a practitioner. They are drawn from our simulated benchmarks and have not been validated on hardware.

- **Rule out a minority or inconsistent fault first.** If the logs a failure implicates fit the model poorly, robust losses or worst-fit removal are cheaper and work (Section 7.10). FACT is for logs that fit well and are wrong.
- **Record the recovery, not only the crash.** A supervisor's takeover is as useful as a crash, provided the record includes it (Section 7.4); a record that stops before the regime where the fault shows is not (Table 6). A correctly labelled ordinary flight that covers that regime serves as the reference, and a record on which the model already fits well, such as hover on the propeller faults, does not (Section 7.5).
- **Read the removed set.** FACT returns the logs it removes. On the quadrotor these were 32–47 flights, including 31–32 of the 32 faulty ones. When it removes a large share of all logs, as on the car, it is acting as a regime filter, a sign that the model cannot represent the plant there. Retraining without the removed logs and measuring the error left on the failure distinguishes the two: in our runs, removing the faulty logs left about 1% of the drone's error on the crash runs and 72–81% of the car's.
- **If the curated model fails again, use that failure.** A second failure fixed the fault that starts mid-flight (Section 7.7).
- **If the cause of the fault is logged, add it as a model input** rather than curating around it, as the battery voltage could be on the Crazyflie.
- **Choose between FACT and learning to reweight by what is needed.** With the failure as its clean set, reweighting completed more dives safely

on all three faults (76–83% against 60–79%), with a smaller margin above the floor, but crashed more on the thrust-gain fault and needs more computation. The controller-cost sweep retains the same direction of tradeoff at all three tested settings (Section 7.3.1). FACT costs one scoring pass and one training, and returns an explicit dataset that can be inspected, versioned and reused by any training pipeline.

8.3. Limitations

All closed-loop results are simulated. The quadrotor plants are fit to real flight logs and the battery fault comes from real logs, but no curated model has flown on hardware, and the mismatch experiment shows that unmodelled effects set a floor that curation cannot lower. The drone and car faults are built by us; the battery fault shows the problem is real, but its cause is withheld rather than unknown. FACT curates whole logs and needs a second failure when a fault enters a log partway. It needs a reference that covers the regime where the model is wrong. Failures can expose that regime without prior identification; other trajectories also work when they cover it. Failures are costly on a real robot; a supervisor's takeover reduces but does not remove that cost. On the car a crude location rule nearly matches it.

The controller-cost sweep covers one fault, five training seeds and three penalty settings, using the existing evaluation task families and task seed. It tests sensitivity rather than independent task generalisation or a complete safety–performance frontier. No tested pair establishes equal risk, and the results do not establish Pareto dominance. The paired intervals, including the three-seed gradient ablation, are exploratory and unadjusted for multiple comparisons; the crash-difference interval at multiplier 0.5 includes zero ($[-0.40, 8.73]$ percentage points).

9. Conclusion

When a consistent fault forms a majority in a regime, a correctly labelled reference from the deployed robot can identify misleading logs that the training data alone cannot distinguish. FACT combines that reference with whole-log attribution and ordinary retraining, returning an inspectable curated dataset without a removal count. One crash dive or a supervisor's takeover fixes the simulated quadrotor's majority propeller fault. The reference, rather than a crash itself, supplies the essential information: a routine flight through the faulty regime also

works. Gradient-free scoring reproduces the thrust-gain curation, while gradient-based scoring improves safe completion on the look-alike fault by 9.3 percentage points.

The comparison with reweighting is a tradeoff. Across three controller-cost settings on thrust gain, FACT has lower mean crash rates and reweighting has higher mean safe completion. Together with the lower measured correction cost and explicit dataset, this supports FACT as a practical curation option, rather than a uniformly superior controller repair. The real-log battery fault requires adding failure data; a model that cannot represent its plant turns curation into a regime filter; and partly faulty logs need a second failure. Hardware evaluation with supervised takeovers and curation of faulty log segments are the next steps.

CRediT authorship contribution statement

Apurba Bose: Conceptualization, Methodology, Software, Formal analysis, Investigation, Visualization, Writing – original draft. **Soovadeep Bakshi:** Conceptualization, Methodology, Software, Formal analysis, Investigation, Visualization, Writing – original draft. **Jiachen Li:** Methodology, Investigation, Writing – review & editing. **Dongmei Chen:** Supervision, Resources, Writing – review & editing.

Declaration of competing interest

The authors declare that they have no known competing financial interests or personal relationships that could have appeared to influence the work reported in this paper.

Funding

This research did not receive any specific grant from funding agencies in the public, commercial, or not-for-profit sectors.

Data availability

The NeuroBEM [7] and NanoBench [38] datasets are publicly available. The code, the flight splits and fault construction, the baseline settings, the per-seed result files, all 2,160 controller-cost evaluation outcomes, and scripts that regenerate the reported numerical summaries are available in a public repository: `https://github.com/soovadeep/few-crash-curation`.

Declaration of generative AI and AI-assisted technologies in the manuscript preparation process

During the preparation of this work the authors used Claude (Anthropic) to search the literature, write and run experiment code, and draft and edit the text, and Codex (OpenAI) to implement and run the controller-cost experiments, analyse the stored results, and revise the manuscript. After using this tool, the authors reviewed and edited the content as needed and take full responsibility for the content of the published article. A script in the code repository recomputes every number in this paper from the stored result files.

References

[1] G. Williams, A. Aldrich, E. A. Theodorou, Model predictive path integral control: From theory to parallel computation, J. Guid. Control Dyn. 40 (2017) 344–357. doi:10.2514/1.G001921.

[2] G. Williams, N. Wagener, B. Goldfain, P. Drews, J. M. Rehg, B. Boots, E. A. Theodorou, Information theoretic MPC for model-based reinforcement learning, in: Proc. IEEE Int. Conf. Robot. Autom. (ICRA), 2017, pp. 1714–1721. doi:10.1109/ICRA.2017.7989202.

[3] A. Nagabandi, G. Kahn, R. S. Fearing, S. Levine, Neural network dynamics for model-based deep reinforcement learning with model-free fine-tuning, in: Proc. IEEE Int. Conf. Robot. Autom. (ICRA), 2018, pp. 7559–7566. doi:10.1109/ICRA.2018.8463189.

[4] K. Chua, R. Calandra, R. McAllister, S. Levine, Deep reinforcement learning in a handful of trials using probabilistic dynamics models, in: Proc. Adv. Neural Inf. Process. Syst. (NeurIPS), 2018.

[5] G. Shi, X. Shi, M. O'Connell, R. Yu, K. Azizzadenesheli, A. Anandkumar, Y. Yue, S.-J. Chung, Neural lander: Stable drone landing control using learned dynamics, in: IEEE International Conference on Robotics and Automation (ICRA), 2019, pp. 9784–9790. doi:10.1109/ICRA.2019.8794351.

[6] G. Torrente, E. Kaufmann, P. Föhn, D. Scaramuzza, Data-driven MPC for quadrotors, IEEE Robotics and Automation Letters 6 (2021) 3769–3776. doi:10.1109/LRA.2021.3061307.

[7] L. Bauersfeld, E. Kaufmann, P. Foehn, S. Sun, D. Scaramuzza, NeuroBEM: Hybrid aerodynamic quadrotor model, in: Proc. Robot.: Sci. Syst. (RSS), 2021. doi:10.15607/RSS.2021.XVII.042.

[8] M. O'Connell, G. Shi, X. Shi, K. Azizzadenesheli, A. Anandkumar, Y. Yue, S.-J. Chung, Neural-Fly enables rapid learning for agile flight in strong winds, Sci. Robot. 7 (2022). doi:10.1126/scirobotics.abm6597.

[9] T. Salzmann, E. Kaufmann, J. Arrizabalaga, M. Pavone, D. Scaramuzza, M. Ryll, Real-time neural MPC: Deep learning model predictive control for quadrotors and agile robotic platforms, IEEE Robotics and Automation Letters 8 (2023) 2397–2404. doi:10.1109/LRA.2023.3246839.

[10] G. Pruthi, F. Liu, S. Kale, M. Sundararajan, Estimating training data influence by tracing gradient descent, in: Proc. Adv. Neural Inf. Process. Syst. (NeurIPS), 2020.

[11] C. Agia, R. Sinha, J. Yang, R. Antonova, M. Pavone, H. Nishimura, M. Itkina, J. Bohg, CUPID: Curating data your robot loves with influence functions, in: Proc. Conf. Robot Learn. (CoRL), volume 305 of *PMLR*, 2025, pp. 2907–2932.

[12] M. Ren, W. Zeng, B. Yang, R. Urtasun, Learning to reweight examples for robust deep learning, in: Proc. Int. Conf. Mach. Learn. (ICML), volume 80 of *PMLR*, 2018, pp. 4334–4343.

[13] L. Brunke, M. Greeff, A. W. Hall, Z. Yuan, S. Zhou, J. Panerati, A. P. Schoellig, Safe learning in robotics: From learning-based control to safe reinforcement learning, Annual Review of Control, Robotics, and Autonomous Systems 5 (2022) 411–444. doi:10.1146/annurev-control-042920-020211.

[14] A. Nagabandi, I. Clavera, S. Liu, R. S. Fearing, P. Abbeel, S. Levine, C. Finn, Learning to adapt in dynamic, real-world environments through meta-reinforcement learning, in: International Conference on Learning Representations (ICLR), 2019.

[15] T. Yu, G. Thomas, L. Yu, S. Ermon, J. Zou, S. Levine, C. Finn, T. Ma, MOPO: Model-based offline policy optimization, in: Proc. Adv. Neural Inf. Process. Syst. (NeurIPS), 2020.

[16] P. J. Huber, Robust estimation of a location parameter, Ann. Math. Statist. 35 (1964) 73–101. doi:10.1214/aoms/1177703732.

[17] B. Han, Q. Yao, X. Yu, G. Niu, M. Xu, W. Hu, I. Tsang, M. Sugiyama, Co-teaching: Robust training of deep neural networks with extremely noisy labels, in: Proc. Adv. Neural Inf. Process. Syst. (NeurIPS), 2018.

[18] C. G. Northcutt, L. Jiang, I. L. Chuang, Confident learning: Estimating uncertainty in dataset labels, J. Artif. Intell. Res. 70 (2021) 1373–1411. doi:10.1613/jair.1.12125.

[19] H. Song, M. Kim, D. Park, Y. Shin, J.-G. Lee, Learning from noisy labels with deep neural networks: A survey, IEEE Transactions on Neural Networks and Learning Systems 34 (2023) 8135–8153. doi:10.1109/TNNLS.2022.3152527.

[20] J. Shu, Q. Xie, L. Yi, Q. Zhao, S. Zhou, Z. Xu, D. Meng, Meta-weight-net: Learning an explicit mapping for sample weighting, in: Proc. Adv. Neural Inf. Process. Syst. (NeurIPS), 2019.

[21] D. Rolnick, A. Ahuja, J. Schwarz, T. Lillicrap, G. Wayne, Experience replay for continual learning, in: Proc. Adv. Neural Inf. Process. Syst. (NeurIPS), 2019.

[22] P. W. Koh, P. Liang, Understanding black-box predictions via influence functions, in: Proc. Int. Conf. Mach. Learn. (ICML), 2017.

[23] A. Ilyas, S. M. Park, L. Engstrom, G. Leclerc, A. Madry, Datamodels: Understanding predictions with data and data with predictions, in: Proc. Int. Conf. Mach. Learn. (ICML), 2022.

[24] S. M. Park, K. Georgiev, A. Ilyas, G. Leclerc, A. Madry, TRAK: Attributing model behavior at scale, in: Proc. Int. Conf. Mach. Learn. (ICML), 2023.

[25] A. Ghorbani, J. Zou, Data Shapley: Equitable valuation of data for machine learning, in: Proc. Int. Conf. Mach. Learn. (ICML), 2019.

[26] J. Li, S. Li, S. Bakshi, J. Xu, D. Chen, IF-CPS: Influence functions for cyber-physical systems—a unified framework for diagnosis, curation, and safety attribution, arXiv preprint arXiv:2603.21543, 2026. doi:10.48550/arXiv.2603.21543.

[27] J. Li, S. Li, S. Bakshi, J. Xu, D. Chen, Influence functions for data attribution in system identification and linear quadratic regulator control, ASME Letters in Dynamic Systems and Control 6 (2026). doi:10.1115/1.4072703.

[28] J. Li, S. Li, D. Chen, Data-attributed adaptive control barrier functions: Safety-certified training data curation via influence analysis, arXiv preprint arXiv:2604.00380, 2026. doi:10.48550/arXiv.2604.00380.

[29] J. Li, S. Li, J. Xu, S. Bakshi, D. Chen, Datamodel-based data selection for nonlinear data-enabled predictive control, arXiv preprint arXiv:2512.00276, 2025. doi:10.48550/arXiv.2512.00276.

[30] J. Li, X. Duan, S. Li, S. Bakshi, D. Chen, DM-MPPI: Datamodel for efficient and safe model path integral control, arXiv preprint arXiv:2512.00759, 2025. doi:10.48550/arXiv.2512.00759.

[31] S. Belkhale, Y. Cui, D. Sadigh, Data quality in imitation learning, in: Proc. Adv. Neural Inf. Process. Syst. (NeurIPS), 2023.

[32] J. Hejna, S. Mirchandani, A. Balakrishna, A. Xie, A. Wahid, et al., Robot data curation with mutual information estimators, in: Robotics: Science and Systems (RSS), 2025. doi:10.15607/RSS.2025.XXI.023.

[33] S. Dass, A. Khaddaj, L. Engstrom, A. Mądry, A. Ilyas, R. Martín-Martín, DataMIL: Selecting data for robot imitation learning with datamodels, arXiv preprint arXiv:2505.09603, 2025. doi:10.48550/arXiv.2505.09603.

[34] D. Gandhi, L. Pinto, A. Gupta, Learning to fly by crashing, in: IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS), 2017, pp. 3948–3955. doi:10.1109/IROS.2017.8206247.

[35] S. Janson, Large deviations for sums of partly dependent random variables, Random Structures & Algorithms 24 (2004) 234–248. doi:10.1002/rsa.20008.

[36] S. S. Srinivasa, P. Lancaster, J. Michalove, M. Schmittle, C. Summers, M. Rockett, R. Scalise, J. R. Smith, S. Choudhury, C. Mavrogiannis, F. Sadeghi, MuSHR: A low-cost, open-source robotic racecar for education and research, arXiv preprint arXiv:1908.08031, 2019. doi:10.48550/arXiv.1908.08031.

[37] E. Todorov, T. Erez, Y. Tassa, MuJoCo: A physics engine for model-based control, in: Proc. IEEE/RSJ Int. Conf. Intell. Robots Syst. (IROS), 2012, pp. 5026–5033. doi:10.1109/IROS.2012.6386109.

[38] S. I. Ullah, J. Baca, NanoBench: A multi-task benchmark dataset for nano-quadrotor system identification, control, and state estimation, arXiv preprint arXiv:2603.09908, 2026. doi:10.48550/arXiv.2603.09908.

Appendix A. Proofs

Proposition 3. Say $f^\star = g_2$. Then $e_t = \rho\Delta(x_t)$, and in expectation a faulty sample has $e_i = -(1-\rho)\Delta(x_i)$ and a correct one $e_i = \rho\Delta(x_i)$. So $\mathbb{E}\phi_i = \rho(1-\rho)\sum_t k\langle\Delta(x_t),\Delta(x_i)\rangle > 0$ for a faulty sample and $-\rho^2\sum_t k\langle\Delta(x_t),\Delta(x_i)\rangle < 0$ for a correct one. The case $f^\star = g_1$ is symmetric.

Normalisation. The expectation of a ratio need not have the sign of the ratio of expectations: a scalar residual with mean 0.5 and noise $+3$ or -1 with probabilities $\frac{1}{4}$ and $\frac{3}{4}$ has $\mathbb{E}[e/|e|] = -\frac{1}{2}$, so its normalised score has the opposite expected sign to its unnormalised one. Proposition 4 rules this out with symmetric noise in two or more dimensions; in one dimension symmetry is not enough, since mean 0.5 and noise uniform on $[-3,-2]\cup[2,3]$ give $\mathbb{E}[e/|e|] = 0$.

Proposition 4. In this setting $\hat{f}$ and the failure residuals e_t are fixed by (i) and (ii), so $a_i = \sum_t k(x_t,x_i)e_t$ does not depend on sample i's label noise; only e_i does. Then $\phi_i = -\langle a_i, e_i\rangle$ and, since $J_iJ_i^\top = c_iI$, $\|J_i^\top e_i\| = \sqrt{e_i^\top J_iJ_i^\top e_i} = \sqrt{c_i}\|e_i\|$, so $\tilde{\phi}_i = -\langle a_i, e_i/\|e_i\|\rangle/\sqrt{c_i}$. Write $e_i = m+\varepsilon$ with $m = \mathbb{E}e_i \neq 0$, $u = m/\|m\|$, $s = \langle\varepsilon,u\rangle$ and r the norm of the rest of ε. Negating the part of ε orthogonal to u leaves its distribution unchanged, so $\mathbb{E}[e_i/\|e_i\|]$ is parallel to u, with component $\mathbb{E}\,h_r(\|m\|+s)$, $h_r(z) = z/\sqrt{z^2+r^2}$. Given r, s is symmetric about zero, and h_r is odd and increasing, so $h_r(\|m\|+s)+h_r(\|m\|-s) \geq 0$, strictly when $r>0$, which holds almost surely for outputs of two or more dimensions. Hence $\mathbb{E}[e_i/\|e_i\|] = \kappa u$ with $\kappa > 0$, and $\mathbb{E}\tilde{\phi}_i = -\kappa\langle a_i, m\rangle/(\sqrt{c_i}\|m\|)$ has the sign of $\mathbb{E}\phi_i = -\langle a_i, m\rangle$.

Proposition 5. Take $\mu > 0$; the other case is symmetric. Let $S = \sum_j(c_j - \mathbb{E}c_j)$, so the mean score is at most zero only if $S \leq -n\mu$. Split the indices into the τ classes $j \bmod \tau$, with n_r elements and sums S_r. By τ-dependence the terms within a class are jointly independent, so $\mathbb{E}e^{-\lambda\tau S_r} \leq e^{\lambda^2\tau^2\sigma^2 n_r/2}$, and the generalised Hölder inequality gives

$$\mathbb{E}e^{-\lambda S} = \mathbb{E}\prod_r e^{-\lambda S_r} \leq \prod_r \left(\mathbb{E}e^{-\lambda\tau S_r}\right)^{1/\tau} \leq e^{\lambda^2\tau\sigma^2 n/2}.$$

Markov's inequality with $\lambda = \mu/(\sigma^2\tau)$ gives

$$P(S \le -n\mu) \le e^{-\lambda n\mu + \lambda^2\sigma^2 n\tau/2} = e^{-n\mu^2/(2\tau\sigma^2)}.$$

A cosine lies in $[-1,1]$, so it is 1-sub-Gaussian by Hoeffding's lemma. The bound over logs is a union bound. A single sample's sign error is a fixed property of its distribution.

Appendix B. Experimental protocol

Quadrotor plant and planner. The state is position, velocity, attitude quaternion and body rates; the plant integrates at 50 Hz. Body rates follow commands with a 40 ms first-order lag, and collective thrust is capped at 14 m/s^2 (thrust-to-weight 1.4). MPPI samples $K = 512$ command sequences over $H = 40$ steps (0.8 s), with Gaussian noise of standard deviation 3.0 m/s^2 on collective thrust and 1.5, 1.5, 1.0 rad/s on the body-rate commands, temperature $\lambda = 1$, and the zero-noise sequence always among the samples. The stage cost is $\|p - p_{\text{goal}}\| + 5\,\text{softplus}(-10(c-0.1)) + 0.02\|\omega_{\text{cmd}}\|^2 + 0.01(T-g)^2$, with c the clearance to the hangar walls, floor and ceiling (minus a 0.2 m body radius) and T the commanded collective thrust. MPPI replans every 3 steps (about 17 Hz) from the state predicted at the time its first new command reaches the motors, 60 ms later. A run lasts at most 5 s.

Tasks. Dives start at rest 4.5 m high and end 0.7 m high, 2–4 m away horizontally (test), 0.5–1.5 m (steep) or 5–8 m (shallow). A run crashes if the clearance ever falls below zero, and reaches the goal if it ever comes within 0.5 m of it, so a dive can reach the goal on its way into the floor (the faulty models' reach is mostly this). The goal is 0.2 m above the clearance limit, and the cost's floor barrier holds a planner with a correct model about 0.45 m above it, at the edge of the radius; reach therefore mostly measures how low the planner settles, and a curated model's reach is best compared with the oracle's. Table 2 re-flies the principal fixes recording each dive, and counts safe completion: reaching the goal without crashing. Attribution dives (task seed 11) and evaluation dives (task seed 7) are disjoint draws.

Measurements and targets. NeuroBEM's processed logs [7] give each flight at 400 Hz, split into segments without motion-capture dropouts, with velocity filtered from motion capture and with body rates and body-frame acceleration filtered from motion capture and the onboard IMU together; the synchronisation and filtering are the dataset authors'. We keep every fourth sample (100 Hz)

and do no further filtering or differentiation. The collective command is $c = k_f \sum_j (\Omega_j/1000)^2$ from the four measured motor speeds Ω_j (rad/s), with k_f fit by least squares to the vertical body-frame acceleration of NeuroBEM's training segments, and the target is the body-frame specific force minus $(0,0,c)$. NanoBench logs at 100 Hz [38]; we use the dataset folder of its public repository (`github.com/syediu/nanobench`, commit 934a1ab, the folder unchanged since the first release), 107 recordings of one Crazyflie 2.1. We normalise the motion-capture quaternion, rotate the motion-capture velocity into the body frame, take the body rates from motion capture, smooth the IMU's specific force with a centred five-sample moving average, and keep airborne rows: more than 0.25 m up, specific force below $3g$ with a vertical component above $0.3g$, and motor commands in range (a few rows carry corrupt values). Recordings with fewer than 2 s of such rows are dropped, leaving 105. The battery voltage, logged at 10 Hz, is held between readings. To test how much FACT depends on these choices, we remade the propeller benchmark's training targets three other ways, with the plant, and so the failure's labels, unchanged: smoothed further by a centred five-sample moving average, shifted one sample (10 ms) late against the inputs, as a synchronisation error, or recomputed by central differences of the world-frame velocity plus gravity, rotated into the body frame. These change the targets by 0.5–12% of their RMS, depending on the axis. Over 5 seeds the sign rule finds 31–32 of the 32 bad flights in every case (AUROC 0.99–1.00), and the planner crashes in $4.4_{\pm 0.8}$, $3.9_{\pm 0.8}$ and $2.5_{\pm 0.9}$% of dives, against $3.1_{\pm 1.2}$% with the dataset's targets.

Failure runs and labels. The faulty model's planner flies the 24 attribution dives in order; the failure runs are the first one or three that crash, cut at the first contact. Each transition's target is recovered from the recorded states and commands by inverting the plant's integration step and subtracting the nominal thrust model, the same map that produced the training targets from the NeuroBEM logs. Section 7.6 adds noise to the recorded states and truncates the record.

Data roles. The training logs are the 95 NeuroBEM flights with their recorded targets, with the fault applied to the targets of the chosen flights. The plant, which flies every failure run and every evaluation, is fit on the same flights with the recorded targets, so the plant and the planner's model share inputs, and share labels everywhere except where the fault applies. Table 6 replaces the plant by other model classes, by an MLP fit on 70% of the flights, and by a plant fit on a held-out third of the flights that the planner's logs exclude; on the Crazyflie the plant is also refit on a held-out third of the flights.

Car. MuJoCo simulates the stock MuSHR model with a 2 ms step, and the planner acts every 50 ms. The state is the planar pose, body-frame velocity, yaw

rate, steering angle and mean wheel speed; the commands are steering (normalised to $[-1,1]$, ± 0.38 rad) and a target speed of at most 2.5 m/s. The learned model takes position (which sets the surface), body-frame velocity, yaw rate, steering angle, wheel speed and the two commands, and predicts the state's change over 50 ms, with displacement and heading change in the body frame. MPPI samples $K = 256$ command sequences over $H = 30$ steps (1.5 s), with Gaussian noise of standard deviation 0.5 on normalised steering and 1.0 m/s on target speed, drawn every 5 steps and interpolated linearly, temperature $\lambda = 1$, replanning every step. The stage cost is the remaining path length to the goal along the corridor plus $30\,\mathrm{softplus}(-10(c-0.25)) + 0.05\,\delta_{\mathrm{cmd}}^2$, with c the clearance to the walls minus a 0.25 m body radius and δ_{cmd} the normalised steering command, and the terminal cost is five times the remaining length. A run lasts at most 10 s; it crashes if the clearance falls below zero and reaches the goal if it comes within 0.5 m of the end of the corridor without crashing. Exploration logs are 1 s of random smooth commands from random states in the corridor; route runs and look-alikes drive the corridor with a pure-pursuit controller.

Crazyflie. Plant and planner have the quadrotor's form at the Crazyflie's scale: 50 Hz, a 40 ms body-rate lag, a 60 ms command delay and replanning every 3 steps. The learned model takes body velocity, body rates and a collective command $c = k_f \sum_j \mathrm{pwm}_j^2$, with one k_f fit on all flights, and predicts the body specific force minus $(0,0,c)$; the plant is a network of the same form that also takes the battery voltage, fit on all flights and flown at 3.1 V. MPPI samples $K = 512$ sequences over $H = 40$ steps (0.8 s), with noise of standard deviation 2.0 m/s^2 on collective thrust and 1.0, 1.0 and 0.5 rad/s on the body rates, and $\lambda = 1$. The stage cost is $\|p - p_{\mathrm{goal}}\| + \frac{5}{3}\,\mathrm{softplus}(-30(c-0.02)) + 0.02\|\omega_{\mathrm{cmd}}\|^2 + 0.01(T-g)^2$, with c the clearance to a $3.2 \times 2.6 \times 1.6$ m box (the NanoBench flight volume) minus a 0.05 m radius. The 24 attribution and 24 evaluation dives (disjoint draws) start at rest 1.4 m above the floor and end 0.12 m above it, up to 1.8 m away horizontally. A run lasts at most 4 s, crashes if the clearance falls below zero, and reaches the goal if it comes within 0.15 m of it. Every model takes as many gradient steps as a fit on all flights.

Training. Every learned model, and the learned drone plants, is an MLP with two hidden layers of 128 SiLU units, trained with Adam (learning rate 2×10^{-3}): on the drones for 12 epochs with batch 2,048 and the rate decayed by 0.8 per epoch (2,640 updates on the full data), on the car for 60 epochs with batch 512 and a cosine schedule. Each seed fixes the network initialisation and batch order of every model.

Timing. One core of an Intel Xeon at 2.1 GHz (PyTorch 2.14, CPU, one

thread), seed 1. A complete fix is timed from the logs and the crash record (states and commands) to a trained model: it includes building the training tensors and the crash dive's labels, any scoring and the whole training, but not flying the faulty model or the evaluation. We report the median of three runs (one for per-sample reweighting). On the propeller, thrust-gain and look-alike benchmarks an ordinary training takes 11 s, FACT 7–8 s (11–12 s with matched updates), the gradient-free score 6–7 s, and learning to reweight 31 s with the look-ahead and 27–28 s with forward-mode products; with per-sample gradients it takes 443 s on the propeller benchmark. That benchmark has 449,714 samples; the MLP has 17,923 parameters, and a full-data training takes 2,640 updates of batch 2,048. Learning to reweight needs every sample's gradient alignment with the clean set at every step: from per-sample gradients (`torch.func.vmap`), from the look-ahead of Ren et al. [12], which differentiates the clean loss after one step with respect to per-sample loss weights at zero, or from one forward-mode product of the per-sample losses along the clean-set gradient. The three give numerically equivalent weights in our checks (largest relative difference 2×10^{-7}, on four batches), but rounding differs, and closed-loop flight amplifies it (Table 2). FACT's scoring takes 39 s with per-sample gradients, or 0.9 s using that a linear layer's per-sample gradient is the outer product of its input and output gradients, so its norm and inner products need one backward pass per batch (scores equal to 3×10^{-5}); the sign rule keeps 57% of the samples (49% with the gradient-free score).

Baseline settings. Huber loss uses $\delta = 1$ in normalised units. Worst-fit removal ranks logs by their mean training loss under the faulty model. The ensemble penalty adds β times the norm of the members' standard deviation (m/s^2) to the cost of every predicted step. CUPID [11] scores a demonstration by its influence on the closed-loop return, $-\nabla J^\top H^{-1} \nabla \ell$, with ∇J estimated by REINFORCE from m rollouts with returns $+1$ for a success and -1 for a failure and no baseline; a demonstration's score is the sum over its samples, and the top k are removed once. We keep all of these, with $m = 100$ rollouts of the faulty model's planner. Our adaptations: the demonstrations are flight logs and, in the REINFORCE form, ℓ is the dynamics model's prediction loss; the identity replaces CUPID's TRAK approximation of H^{-1}, as for every score in this paper; and since MPPI has no action likelihood, the REINFORCE form adds Gaussian exploration noise, 0.5 times MPPI's sampling noise, to each executed command, so that $\nabla \log \pi$ is the noise over its variance times the gradient of the MPPI command, differentiated through one planning step with the state held fixed rather than through the closed loop. The label-free form, the one Agia et al. [11] use in practice, replaces both $\nabla \log \pi$ and $\nabla \ell$ by the gradient of the squared network output, on rollout and train-

ing samples alike. Scoring flights by their mean instead of the sum gives the same AUROCs within 0.05.